

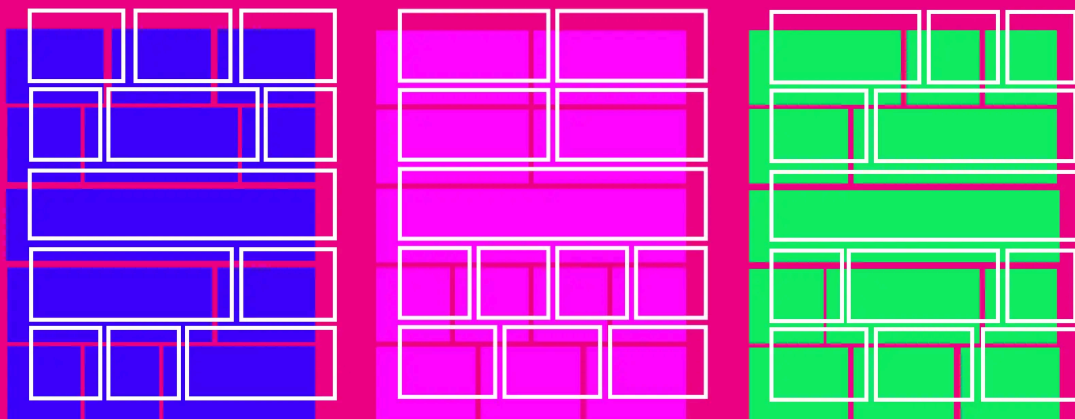
XCM: The Cross-Consensus Message Format

This primer on XCM, Polkadot's inter-chain messaging format is the first in a series of articles explaining what it does, how it works, how to use it, and what it could become.

Product & Engineering



By Dr. Gavin Wood • September 6, 2021



*As the final Polkadot 1.0 release, complete with Parachains draws close, the Cross-Consensus Messaging format, **XCM** for short, is approaching its first*



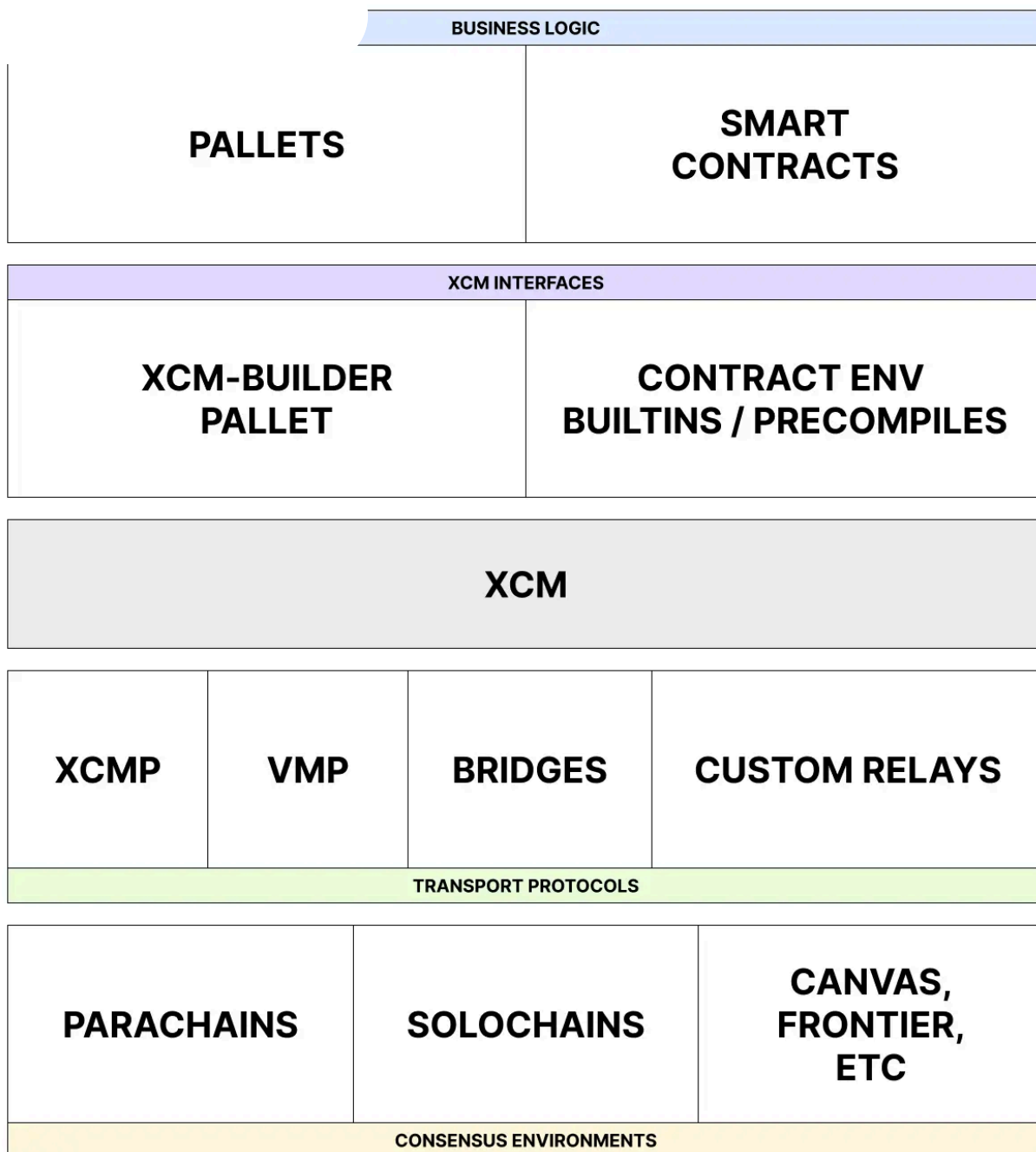
production-ready release This is an introduction to the format, its goals, how it
solves typical cross-chain tasks.

One fun fact to begin with... XCM is the “cross-consensus” messaging format, rather than just “cross-chain”. This difference is a sign of the goals of the format which is designed for communicating the kinds of ideas sent not just between chains, but also smart-contracts and pallets, and over bridges and sharded enclaves like Polkadot’s Spree.

👉 A Format, not a Protocol

To understand XCM better, it’s important to understand its boundaries and where it fits in the Polkadot technology stack. XCM is a messaging *format*. It is not a messaging *protocol*. It cannot be used to actually “send” any message between systems; its utility is only in expressing what should be done by the receiver.

Not including bridges and the contracts pallet, Polkadot comes with three distinct systems for actually communicating XCM messages between its constituent chains: UMP, DMP and XCMP. UMP (Upward Message Passing) allows parachains to send messages to their relay chain. DMP (Downward Message Passing) allows the relay chain to pass messages down to one of their parachains. XCMP, is perhaps the best known of them, and this allows the parachains to send messages between themselves. XCM can be used to express the meaning of the messages over each of these three communication channels.



In addition to sending messages between chains, XCM is also useful in other contexts, for transacting with a chain whose transaction format you don't necessarily know well in advance. With chains whose business logic changes little (for example Bitcoin), the transaction format – or the format used by wallets to send instructions to the chain – tends to remain exactly the same, or at least compatible, indefinitely. With highly evolvable metaprotocol-based chains such as Polkadot and its constituent parachains, the business logic can be upgraded across the network with a single transaction. This can change anything, including

the transaction format introducing a potential problem for wallet maintainers, $\Rightarrow$ required to be kept offline (such as Parity Signer). Since XCM is well-referenced, abstract and general, it can be used as a means of providing a long-lasting transaction format for wallets to use to create many common transactions.

Goals

XCM aims to be a *language communicating ideas between consensus systems*. It should be general enough for it to be properly useful throughout a growing ecosystem. It should be extensible. Since the extensibility will inevitably imply change, it should also be future-proof and forwards-compatible. Finally, it should be efficient enough to run on-chain, and possibly in a metered environment.

Like all languages, some individuals will tend to use some elements more than others. XCM is not designed in such a way that every system which supports XCM is expected to be able to interpret any possible XCM message. Some messages will not have reasonable interpretations under some systems. Others might be reasonable, but still intentionally unsupported by the interpreter owing to resource constraints or because the same content can be expressed in a clearer and more canonical manner. Systems will inevitably only support a subset of possible messages. Heavily resource-constrained systems (like smart contracts) may support only a very limited “dialect”.

This generality extends even as far as concepts like payment of fees for executing the XCM message. Since we know that XCM may be used on diverse systems including a gas-metered smart contract platform and community parachains all the way to trusted interactions between system parachains and their relay chain, we do not want to bake elements such as fee payment too deep and irreversibly in the protocol.



Why don't we just use the native message format?

Piggybacking on the native message/transaction format of a chain or smart contract can be useful in certain circumstances, but does have some big drawbacks that make it less useful for the goals of XCM. Firstly, there is a lack of compatibility between chains, so a system which intends to send messages to more than one destination would need to understand how to author a message for each. On that note, even a single destination may alter its native transaction/message format over time. Smart contracts might get upgrades, blockchains might introduce new features or alter existing ones and in doing so change their transaction format.

Secondly, common use-cases on chains do not easily fit into a single transaction; special tricks may be required to withdraw funds, exchange them and then deposit the result all inside a single transaction. Onward notifications of transfers, needed for a coherent reserve-asset framework, do not exist in chains unaware of others.

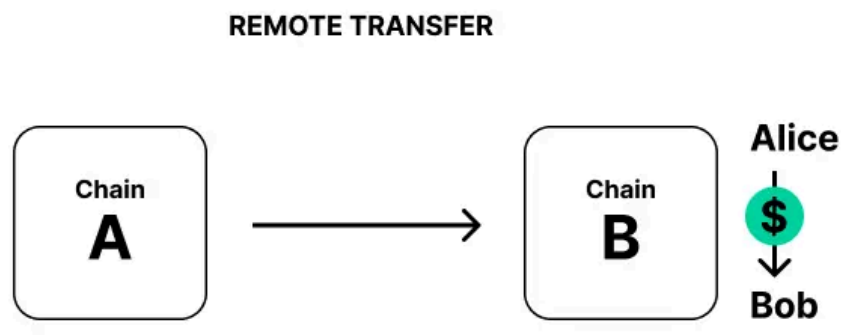
Thirdly, operations such as the payment of fees do not easily fit into a model which assumes fee-payment has already been negotiated like smart contract messages. Transaction envelopes, in comparison, provide some system for payment of processing, but are also generally designed to contain a signature which is not something that makes sense when communicating between consensus systems.



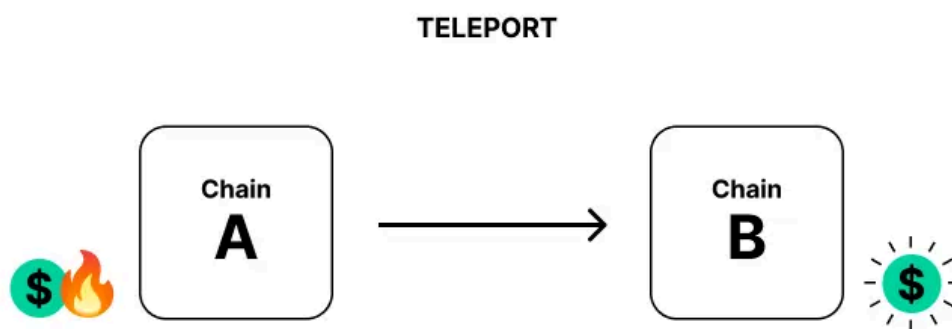
Some Initial Use-cases

While the goal of XCM is to be general, flexible and future-proof, there are of course practical needs which it must address, not least the transfer of tokens between chains. The optional payment of fees (perhaps using those tokens) is another, as is a general interface for conducting an exchange service, common throughout the DeFi world. Finally, it should be possible to use the XCM language to conduct some platform-specific action; for example, within a Substrate chain, it can be desirable to dispatch a remote call into one of its *pallets* to access a niche feature.

On top of that, there are many models for transferring tokens which we would not want to simply control an account on a remote chain, allowing the local chain to have an address on the remote chain for receiving funds and to eventually transfer those funds it controls into other accounts on that remote chain.



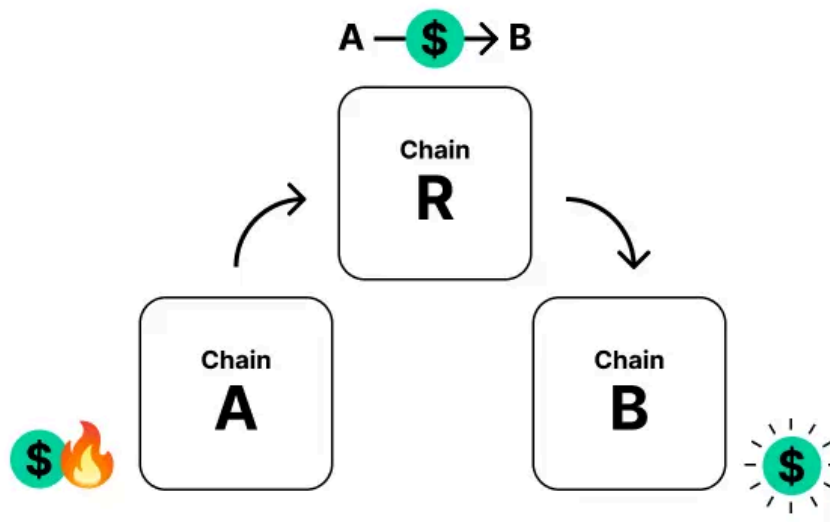
We might have two consensus systems, both of which are native homes for a particular token. Imagine a token such as USDT or USDC, which has instances — all perfectly fungible— on several different chains. It should be possible to burn such a token on one chain and mint a corresponding token on another supported chain. In the parlance of XCM, we call this *teleporting* owing to the idea that the apparent movement of an asset in fact happens by destroying it on one side and creating a clone on the other side.



Finally, there may be two chains which want to nominate a third chain, one on which an asset might be considered native, to be used as a reserve for that asset. The derivative form of the asset on each of those chains would be *fully backed*, allowing the derivative asset to be exchanged for the underlying asset on the reserve chain backing it. This might be the case where the two chains do not necessarily trust each other, but (at least as far as the asset in question is

concerned) are willing to trust the native chain of the asset. An example here is the Polkadot ecosystem of parachains which would like to send DOT between each other. They each have a local form of DOT which is backed fully by DOT controlled by the parachain on the Statemint chain (a native hub for DOT). When the local form of DOT is sent between the chains, in the background the “real” DOT is moving between parachain accounts on Statemint.

RESERVE-BASED TRANSFER



Even this apparently modest level of functionality has a relatively large number of configurations whose usage might be desirable and requires some interesting design to avoid overfitting.

👁 The Anatomy of XCM

At the core of the XCM format lies the *XCVM*. Contrary to how it might look to some, this is not a (valid) roman numeral (though if it were, it'd probably mean 905). In fact this stands for Cross-Consensus Virtual Machine. It's an ultra-high level non-Turing-complete computer whose instructions are designed to be roughly at the same level as transactions.

A “message” in XCM is actually just a programme that runs on the XCVM. It is one or more XCM *instructions*. The programme executes until it either runs to the end

or hits an error at which point it finishes up (I'm leaving that intentionally

3.

The XCVM includes a number of *registers*, as well as access to the overall state of the consensus system which is hosting it. Instructions might change a register, they might change the state of the consensus system or both.

One example of such an instruction would be `TransferAsset` which is used to transfer an asset to some other address on the remote system. It needs to be told which asset(s) to transfer and to whom/where the asset is to be transferred. In Rust, it is declared like this:

```
enum Instruction {  
    TransferAsset {  
        assets: MultiAssets,  
        beneficiary: MultiLocation,  
    }  
    /* snip */  
}
```

As you might guess, `assets` is the parameter which expresses which assets are to be transferred, and `beneficiary` states to whom/where they are to be put. We are of course missing one other piece of information, namely *from whom/where the assets are to be taken*. This is automatically inferred from the *Origin Register*. When the programme begins, this register is generally set according to the transport system (the bridge, XCMP or whatever) to reflect where the message actually came from, and it is the same type of information as the `beneficiary`. The Origin Register operates as a protected register – the programme cannot set it arbitrarily though there are two instructions which can be used to alter it in certain ways.

The types used are quite fundamental ideas in XCM: assets, represented by `MultiAsset` and locations-within-consensus, represented by `MultiLocation`. The Origin Register is an optional `MultiLocation` (optional, because it can be cleared entirely if desired).

Location in XCM

The `Location` type identifies any single *location* that exists within the world of consensus. It is quite an abstract idea and can represent all manner of things that exist within consensus, from a scalable multi-shard blockchain such as Polkadot all the way down to a lowly ERC-20 asset account on a parachain. In computer science terms, it's really just a global singleton data structure, regardless of its size or complexity.

`MultiLocation` always expresses a location *relative to the current location*. You can think of it a bit like a file system path but where there is no way of directly expressing the “root” of the file system tree. This is for a simple reason: In the world of Polkadot, blockchains can be merged into, and split from, other blockchains. A blockchain can begin life very much alone, and eventually be elevated to become a parachain within a larger consensus. If it did that, then the meaning of “root” would change overnight and this could spell chaos for XCM messages and anything else using `MultiLocation`. To keep things simple, we exclude this possibility altogether.

Locations in XCM are hierarchical; some places in consensus are wholly encapsulated within other places in consensus. A parachain of Polkadot exists wholly within the overall Polkadot consensus and we call it an *interior location*. Putting it more strictly, we can say that whenever there is a consensus system any change in which implies a change in another consensus system, then the former system is *interior* to the latter. For example, a `Canvas` smart contract is *interior* to the contracts pallet which hosts it. An UTXO in Bitcoin is interior to the Bitcoin blockchain.

This means that XCM doesn't distinguish between the two questions “who?” and “where?”. From the point of view of something fairly abstract like XCM, the difference isn't really important — the two blur and become essentially the same thing.

`MultiLocations` are used to identify places to send XCM messages, places which can receive assets and then can even help describe the type of an asset itself, as we will see. Very useful things.

When written down in text like this article, they are expressed as some number of (parent consensus system) components followed by some number of junctions, all separated by /. (This isn't what generally happens when we express them in a language like Rust, but it makes sense in writing since it's quite a lot like the familiar directory paths that are in widespread use.)

Junctions identify an interior location within its encapsulating consensus system. If there are no parents/junctions at all, then we just say that the location is *Here*.

Some examples:

- `../Parachain(1000)`: Evaluated within a parachain, this would identify our sibling parachain of index 1000. (In Rust we would write `ParentThen(Parachain(1000)).into()`.)
- `../AccountId32(0x1234...cdef)`: Evaluated within a parachain, this would identify the 32-byte account `0x1234...cdef` on the relay chain.
- `Parachain(42)/AccountKey20(0x1234...abcd)`: Evaluated on a relay chain, this would identify the 20-byte account `0x1234...abcd` on parachain number 42 (presumably something like Moonbeam which hosts Ethereum-compatible accounts).

There are many different types of *junction* for identifying places you might find on-chain in all sorts of ways such as keys, indices, binary blobs and plurality descriptions.

Assets in XCM

When working in XCM it's often needed to refer to an asset of some sort. This is because practically all public blockchains in existence rely on some native digital asset to provide the backbone for its internal economy and security mechanism. For Proof-of-Work blockchains such as Bitcoin, the native asset (BTC) is used to reward the miners who grow the blockchain and prevent double-spending. For Proof-of-Stake blockchains such as Polkadot, the native asset (DOT) is used as a form of collateral, where network keepers (known as *stakers*) must risk it in order to generate valid blocks and be rewarded in kind.

Some blockchains manage multiple assets, e.g. Ethereum's ERC-20 framework has tokens to be managed on-chain. Some manage assets which are not fungible such as Ethereum's ETH but rather are non-fungible — one-of-a-kind instances; Crypto-kitties was an early example of such non-fungible tokens or NFTs.

XCM is designed to be able to handle all such assets without breaking a sweat. For this purpose there is the datatype `MultiAsset` together with its associated types `MultiAssets`, `WildMultiAsset` and `MultiAssetFilter`. Let's look at `MultiAsset` in Rust:

```
struct MultiAsset {  
    id: AssetId,  
    fun: Fungibility,  
}
```

So there's two fields which define our asset: `id` and `fun`, this is pretty indicative of how XCM approaches assets. Firstly, an overall asset identity must be provided. For fungible assets this simply identifies the asset. For NFTs this identifies the overall asset “class” — different asset instances may within this class.

```
enum AssetId {  
    Concrete(MultiLocation),  
    Abstract(BinaryBlob),  
}
```

The asset identity is expressed in one of two ways; either *Concrete* or *Abstract*. *Abstract* is not really in use, but it allows asset IDs to be specified by name. This is convenient, but relies on the receiver interpreting the name in the way that the sender expects which may not always be so easy. *Concrete* is in general usage and uses a *location* to identify an asset unambiguously. For native assets (such as DOT), the asset tends to be identified as the chain which mints the asset (the Polkadot Relay Chain in this case, which would be the location `..` from one of its parachains). Assets that are primarily administered within a chain's pallet may be identified by a location including their index within that pallet. For example, the Karura parachain might refer to an asset on the Statemine parachain with the location `../Parachain(1000)/PalletInstance(50)/GeneralIndex(42)`.

```

        t),
        nonFungible(AssetInstance),
    }

```

Secondly, they must be either fungible or non-fungible. If they're fungible, then there should be some associated *non-zero* amount. If they're not fungible, then instead of an amount, there should be some indication of which instance they are. This is commonly expressed with an index, but XCM also allows various other datatypes to be used such as arrays and binary blobs.

This covers `MultiAsset`, but there are three other associated types that we sometimes use. `MultiAssets` is one of them and really just means a set of `MultiAsset` items. Then we have `WildMultiAsset`; this is a wildcard which can be used to match against one or more `MultiAsset` items. There are actually only two kinds of wildcard that it supports: `All` (which matches against all assets) and `Allow` which matches against all assets of a particular identity (`AssetId`) and fungibility. Notably, for the latter, the amount (in the case of fungibles) or instance(s) (for non-fungibles) does not need to be specified and all are matched.

Finally, there is `MultiAssetFilter`. This is used most often and is really just a combination of `MultiAssets` and `WildMultiAsset` allowing either a wildcard or a list of *definite* (i.e. not wildcard) assets to be specified.

In the Rust XCM API, we provide a lot of conversions to make working with these datatypes as painless as possible. For example, to specify the fungible `MultiAsset` which equals 100 indivisible units of the DOT asset (*Planck*, for those in the know) when we are on the Polkadot Relay Chain, then we would use `(Here, 100).into()`.

👉 The Holding Register

Let's take a look at another XCM instruction: `WithdrawAsset`. On the face of it, this is a bit like the first half of `TransferAsset`: it withdraws some assets from the account of the place specified in the Origin Register. But what does it do with

them? — if they don't get deposited anywhere then it's surely a pretty useless
t declaration:

```
WithdrawAsset(MultiAssets),
```

So, there's only one parameter this time (of type `MultiAssets` and which dictates which assets must be withdrawn from the ownership of the Origin Register). But there is no location specified in which to put the assets.

The withdrawn and unspent assets are temporarily held in what is known as the *Holding Register* – (“holding” because they’re in a temporary position which cannot persist indefinitely). There are a number of instructions which operate on the Holding Register. One very simple one is the `DepositAsset` instruction. Let's take a look at it:

```
enum Instruction {  
    DepositAsset {  
        assets: MultiAssetFilter,  
        max_assets: u32,  
        beneficiary: MultiLocation,  
    },  
    /* snip */  
}
```

Aha! The astute reader will see that this looks rather like the missing half of the `TransferAsset` instruction. We have the `assets` parameter which specifies which of the assets should be removed from the Holding Register to be deposited on-chain. `max_assets` lets the XCM author inform the receiver how many unique assets are intended to be deposited. (This is helpful when calculating fees in advance of knowing the contents of the Holding Register since depositing an asset can be a costly operation.) Finally there is the `beneficiary`, which is the same parameter we met earlier in the `TransferAsset` operation.

There are many instructions which express actions to do on the Holding Register, and `DepositAsset` is one of the simplest. Some others are rather more sophisticated 😊.



Fee payment in XCM

Fee payment in XCM is a rather important use-case. Most parachains in the Polkadot community will require their interlocutors to pay their way for any operations that they wish to conduct, lest they leave themselves open to “transaction spam” and a denial-of-service attack. Exceptions to this exist when chains have good reason to believe that their interlocutor will be well-behaved—this is the case when the Polkadot Relay Chain corresponds with the Polkadot Statemint common-good chain. However for the general case, fees are a good way of ensuring that XCM messages and their transport protocols cannot be over-used. Let’s look at how fees can be paid when XCM messages arrive into Polkadot.

As already mentioned, XCM does not include the idea of fees and fee-payment as a *first-class citizen*: Unlike, say, the Ethereum transaction model, fee payment is not something baked into the protocol that use-cases which have no need for must conspicuously circumvent. Like Rust with its zero-cost abstractions, fee payment comes with no great design overhead in XCM.

For systems that do require some fee payment though, XCM provides the ability to buy execution resources with assets. Doing so, broadly speaking, consists of three parts:

- Firstly, some assets need to be provided.
- Secondly, the exchange of assets for compute time (or *weight*, in Substrate parlance) must be negotiated.
- Finally, the XCM operations will be performed as instructed.

The first part is managed by one of a number of XCM instructions which provide assets. We already know one of these (`WithdrawAsset`), but there are several others which we will see later. The resultant assets in the Holding Register will of course be used for paying fees associated with executing the XCM. Any assets not used to pay fees we will be depositing in some destination account. For our example, we’ll assume that the XCM is happening on the Polkadot Relay Chain and that it’s for 1 DOT (which is 10,000,000,000 indivisible units).

So far our XCM instruction looks like:

```
000_000_000).into()),
```

This brings us to the second part, exchanging (some of) these assets for compute time to pay for our XCM. For this we have the XCM instruction `BuyExecution`. Let's take a look at it:

```
enum Instruction {
    /* snip */
    BuyExecution {
        fees: MultiAsset,
        weight: u64,
    },
}
```

The first item `fees` is the amount which should be taken from the *Holding Register* and used for fee-payment. It's technically just the maximum since any unused balance is immediately returned.

The amount that ends up being spent is determined by the interpreting system — `fees` only limits it and if the interpreting system needs to be paid more for the execution desired, then the `BuyExecution` instruction will result in error. The second item specifies an amount of execution time to be purchased. This should generally be no less than the weight of the XCM programme in total.

In our example we'll assume that all XCM instructions take a million weight, so that's two million for our two items so far (`WithdrawAsset` and `BuyExecution`) and a further one for what's coming next. We'll just use all the DOT that we have to pay those fees (which is only a good idea if we trust the destination chain not to have crazy fees — we'll assume that we do). Let's have a look at our XCM so far:

```
WithdrawAsset((Here, 10_000_000_000).into()),
BuyExecution {
    fees: (Here, 10_000_000_000).into(),
    weight: 3_000_000,
},
```

The third part of our XCM comes in depositing the funds remaining in the Holding Register using the `DepositAsset` instruction. We don't actually know how much is remaining in the Holding Register, but that doesn't matter since we can specify a wildcard for the asset(s) which should be deposited. We'll place them in the sovereign account of Statemint (which is identified as `Parachain(1000)`).

Our final XCM instruction therefore looks like this:

```
WithdrawAsset((Here, 10_000_000_000).into()),  
BuyExecution {  
  fees: (Here, 10_000_000_000).into(),  
  weight: 3_000_000,  
},  
DepositAsset {  
  assets: All.into(),  
  max_assets: 1,  
  beneficiary: Parachain(1000).into(),  
},
```

⌘ Moving Assets between Chains in XCM

Sending an asset to another chain is probably the most common use-case for inter-chain messaging. Allowing one chain to administer another chain's native asset allows for all sorts of derivative use-cases (no pun intended), the simplest being a decentralised exchange but generally grouped together as decentralised finance or DeFi.

Generally speaking there are two ways that assets move between chains and this depends on whether the chains trust each other's security and logic or not.

✨ **Teleporting**

For chains that trust each other (such a homogeneous shards under the same relay chain umbrella), we can use a framework that Polkadot calls *teleporting*, which basically just means destroying an asset on the sending side and minting it on the receiving side. This is simple and efficient — it only requires the coordination of the two chains and only involves one action on either side. Unfortunately, if the receiving chain cannot 100% trust the sending chain to actually destroy the asset which it is minting (and indeed not to mint assets outside of the agreed rules for the asset), then the sending chain really has no basis for minting the asset on the back of a message.

Let's look at how the XCM would look which teleported (most of) 1 DOT from the Polkadot Relay Chain to its sovereign account on Statemint. We'll assume that the fees are already paid on the Polkadot side.

```
WithdrawAsset((Here, 10_000_000_000).into()),
InitiateTeleport {
  assets: All.into(),
  dest: Parachain(1000).into(),
  xcm: Xcm(vec![
    BuyExecution {
      fees: (Parent, 10_000_000_000).into(),
      weight: 3_000_000,
    },
    DepositAsset {
      assets: All.into(),
      max_assets: 1,
      beneficiary: Parent.into(),
    },
  ]),
}
```

As you can see, this looks fairly similar to the straight withdraw-buy-deposit pattern that we saw last. The difference is the `InitiateTeleport` instruction which is inserted around the last two instructions (`BuyExecution` and `DepositAsset`). Behind the scenes, the sender (Polkadot Relay) chain is creating a whole new message when it executes the `InitiateTeleport` instruction; it takes the `xcm` field and places it inside a new XCM, `ReceiveTeleportedAsset`, and sends this XCM on to the receiver (Statemint) chain. Statemint trusts the

Polkadot Relay Chain to have destroyed the 1 DOT on its side prior to sending the

The **beneficiary** is stated as `Parent.into()`, and an astute reader might be wondering what this could refer to in the context on the Polkadot Relay Chain. The answer would be “nothing”, but there is no mistake here. Everything in the `xcm` parameter is written from the perspective of the receiving side, so despite this being a part of the overall XCM which is fed into the Polkadot Relay Chain, it is only actually *executed* on Statemint, and thus it is in Statemint’s context which is it written.

When Statemint eventually gets the message, it looks like this:

```
ReceiveTeleportedAsset((Parent, 10_000_000_000).into()),
BuyExecution {
  fees: (Parent, 10_000_000_000).into(),
  weight: 3_000_000,
},
DepositAsset {
  assets: All.into(),
  max_assets: 1,
  beneficiary: Parent.into(),
},
```

You might notice that this looks rather similar to the previous `WithdrawAsset` XCM. The only major difference is that rather than funding the fees and deposit through a withdrawal from a local account, it is being “magicked” into existence by trusting that the DOT was faithfully destroyed on the sending (Polkadot Relay Chain) side and honouring the `ReceiveTeleportedAsset` message.

Notably, the asset identifier of the 1 DOT we sent on Polkadot Relay Chain (Here, referring to the Relay Chain itself the native home for DOT) has been automatically mutated into its representation on Statemint: `Parent.into()`, which is the Relay Chain’s location from Statemint’s context.

The **beneficiary** is specified as the Polkadot Relay Chain also and so its sovereign account (on Statemint) is credited with newly minted 1 DOT minus fees.

The XCM might just have easily named an account or other place for the `TransferAsset` sent from the Relay Chain could be used to move the DOT.

Reserves

The alternative way to transfer assets across chains is slightly more complicated. A third-party is used known as the *reserve*. The name comes from reserve banking, where assets are held “in reserve” to give credibility to the idea that some issued promise is valuable. For example, if we can reasonably believe exactly 1 “real” (e.g. Statemint or Relay Chain) DOT is redeemable for each “derivative” DOT issued on an independent parachain, then we can treat the parachain’s DOT as being economically equivalent to real DOT, (most banks do something called *fractional*/reserve banking, which means they keep less than the face-value in reserve). This works fine until too many people wish to redeem, and then everything can go quite wrong quite fast.

So, the reserve is the place which stores the “real” assets and, for the purposes of transferral, whose logic and security is trusted by both sender and receiver. Any corresponding assets on the sender and receiver side would then be *derivatives*, but they would be backed with the “real” reserve asset 100%. Assuming that the parachain behaved well (i.e. that it was bug-free and its governance didn’t decide to run off with the reserve), this would make the derivative DOT more or less of the same value as the underlying reserve DOT. The reserve assets are held in the sender/receiver’s *sovereign account* (i.e. the account controllable by the sender or receiver chain) on the reserve chain, so there’s good reason that unless something went wrong with the parachain, they’d be well guarded.

Back to the transfer mechanism, the sender would instruct the reserve to move the assets which the sender owns (and uses as a reserve for its own version of the same asset) into the receiver’s sovereign account, and the reserve – *not the sender!* – informs the receiver about their new credit. This means that the sender and receiver do not need to trust each other’s logic or security, but only that of the chain used as the reserve. However it does imply that the three sides need to coordinate, which increases the overall cost, time and complexity.

Let's look at the XCM required. This time we'll be sending 1 DOT from parachain 2000 to use reserve-backed DOT on parachain 1000. Again, we assume the fees are already paid on the sender side.

```
WithdrawAsset((Parent, 10_000_000_000).into()),
InitiateReserveWithdraw {
  assets: All.into(),
  dest: ParentThen(Parachain(1000)).into(),
  xcm: Xcm(vec![
    BuyExecution {
      fees: (Parent, 10_000_000_000).into(),
      weight: 3_000_000,
    },
    DepositReserveAsset {
      assets: All.into(),
      max_assets: 1,
      dest: ParentThen(Parachain(2001)).into(),
      xcm: Xcm(vec![
        BuyExecution {
          fees: (Parent, 10_000_000_000).into(),
          weight: 3_000_000,
        },
        DepositAsset {
          assets: All.into(),
          max_assets: 1,
          beneficiary: ParentThen(Parachain(2000)).into(),
        },
      ]),
    },
  ]),
},
```

This is a little more complex, as promised. Let's walk through it. The outer part deals with extracting the 1 DOT on the sender side (parachain 2000) and withdrawing the corresponding 1 DOT held on Statemint (parachain 1000) — it uses `InitiateReserveWithdraw` for this purpose and it's pretty self-explanatory.

```
WithdrawAsset((Parent, 10_000_000_000).into()),
InitiateReserveWithdraw {
```

```

        assets: All.into(),
                                achain(1000)).into(),

    }

```

Now we have 1 DOT in the Holding Register on Statemint. Before we can do anything else, we need to buy some execution time on Statemint. This, again, looks pretty familiar:

```

/*snip*/
    xcm: Xcm(vec![
        BuyExecution {
            fees: (Parent, 10_000_000_000).into(),
            weight: 3_000_000,
        },
        DepositReserveAsset {
            assets: All.into(),
            max_assets: 1,
            dest: ParentThen(Parachain(2001)).into(),
            xcm: /* snip */
        },
    ]),
/*snip*/

```

We're using our 1 DOT to pay for fees, and we're assuming one million per XCM operation. With that one operation paid for, we deposit the 1 DOT (minus fees, and we're lazy so we just use `All.into()`) into the sovereign account for parachain 2001, but do so as a *reserve asset* meaning that we also require Statemint to send a notification XCM to that receiving chain informing it of the transfer along with some instructions to be executed on the resulting derivate assets.

`DepositReserveAsset` instructions don't always make much sense; for it to make sense, the `dest` must be a location which can reasonably hold funds on the reserve chain, but also one to which the reserve chain can send an XCM. Sibling parachains happen to fit the bill perfectly.

```

/*snip*/
    xcm: Xcm(vec![
        BuyExecution {

```

```

        fees: (Parent, 10_000_000_000).into(),
            0_000,

    DepositAsset {
        assets: All.into(),
        max_assets: 1,
        beneficiary: ParentThen(Parachain(2000)).into(),
    },
]),
/*snip*/

```

The final part defines part of the message which arrives at parachain 2001. Like with initiating a teleport operation, `DepositReserveAsset` composes and sends a new message in this case `ReserveAssetDeposited`. It is this message, albeit containing the XCM programme which we define, which arrives at the receiving parachain. It will look something like this:

```

ReserveAssetDeposited((Parent, 10_000_000_000).into()),
BuyExecution {
    fees: (Parent, 10_000_000_000).into(),
    weight: 3_000_000,
},
DepositAsset {
    assets: All.into(),
    max_assets: 1,
    beneficiary: ParentThen(Parachain(2000)).into(),
},

```

(This assumes that no fees were actually taken on Statemint and the whole 1 DOT made it over. That's not especially realistic, so the `assets` line is probably going to have a lower number.)

Most of the message should look pretty familiar; the only significant difference with the `ReceiveTeleportedAsset` message we saw in the last section is the top-level instruction `ReserveAssetDeposited`, which fulfills a similar purpose, only rather than meaning “the sending chain burned assets so you can mint equivalent assets”, it means “the sending chain received assets and is holding them for you in reserve, so you can mint fully-backed derivatives”. Either way the

destination chain mints them into the Holding Register, and we deposit them in
it on the receiving chain. 🎉

🏁 Conclusion

That's it for this article; I hope it was helpful in explaining what XCM is and the basics of how it is designed to work. In the next article(s?), we will take a deeper look into the XCVM's architecture, its execution model and its error handling, XCM's versioning system and how upgrades to the format can be managed in a well-connected interdependent ecosystem, as well as its query-response system and how XCM works in Substrate. We will also discuss some of the future directions of XCM, planned features and the process for evolving it.

Read [Part II: Versioning & Compatibility](#)

Read [Part III: Execution & Error Management](#)



GET STARTED

DOT Token
Polkadot Wallets
Staking
Dapps
Bridges

PLATFORM

Polkadot SDK
Polkadot Chain
Polkadot DAO
Polkadot Hub
Case Studies
Use Cases
Papers

DEVELOPERS

Grants & Funding
Courses
Documentation ↗
Blockchain Academy
Parity Bug Bounty ↗
Bridges Bug Bounty

COMMUNITY

[About](#)

[Blog](#)

[Newsroom](#)

[Decentralized Jobs](#)

[Ambassador Program](#)

[Events](#)

[Brand Hub](#)

[Contact](#)

[Polkadot.com](#) [GitHub](#) ↗

SOLUTIONS

[AI](#)

[Business](#)

[DeFi](#)

[DePIN](#)

[Gaming](#)

[Government](#)

[RWAs](#)